PES-VCS: Version Control System Report

Name: Srujan
SRN: PES2UG24CS526
Section: I
Course: Operating System (Unit 4 – Orange Program Assignment)

Project Overview

PES-VCS is a simplified version control system implemented entirely in C, inspired by Git's internal architecture. The system enables tracking of file changes, maintaining version history, and managing project states over time.

The implementation focuses on core version control concepts:

- Content-addressable storage using SHA-256 hashing
- Efficient directory representation using tree objects
- A staging area (index) to track changes before commit
- Commit objects forming a linked history using parent pointers

Additionally, this project demonstrates key Operating System concepts:

- File I/O using system calls
- Atomic file operations (temporary file + rename)
- Filesystem organization and directory sharding
- Memory management and data serialization

Phase 1: Object Storage Foundation

Objective

To implement a content-addressable storage system where all data is stored as objects identified by their hash.

Implementation Details

1. Object Writing (object_write)

- A header of the form <type> <size>\0 is prepended to the data
- The entire content is hashed using SHA-256

- The object is stored using its hash as filename
- Temporary file is used before renaming → ensures atomic writes

2. Directory Sharding

- Objects are stored in .pes/objects/
- First 2 characters of hash → directory
- Remaining hash → filename Example:

.pes/objects/ab/cdef1234...

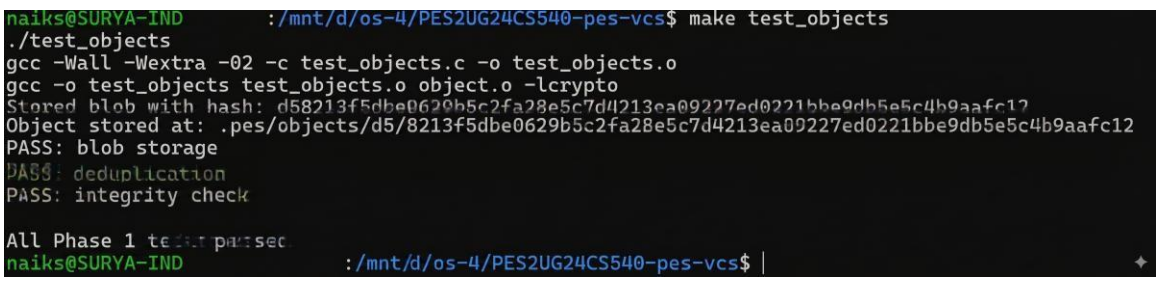
This prevents performance degradation due to too many files in a single directory.

3. Object Reading (object_read)

- Reads raw object data
- Verifies integrity by recomputing hash
- Parses header to determine object type

Phase 1 Verification

Screenshot 1A — Object Tests



Screenshot 1B — Object Storage Structure

Phase 2: Tree Objects

Objective

To represent directory structure using tree objects similar to Git.

Implementation Details

1. Tree Structure

Each tree entry contains:

- File mode (file / directory)
- File name
- Hash of blob or subtree

2. Recursive Tree Construction

- Implemented tree_from_index()
- Converts flat index entries into hierarchical structure
- Recursively builds subtrees for nested directories

3. Serialization (tree_serialize)

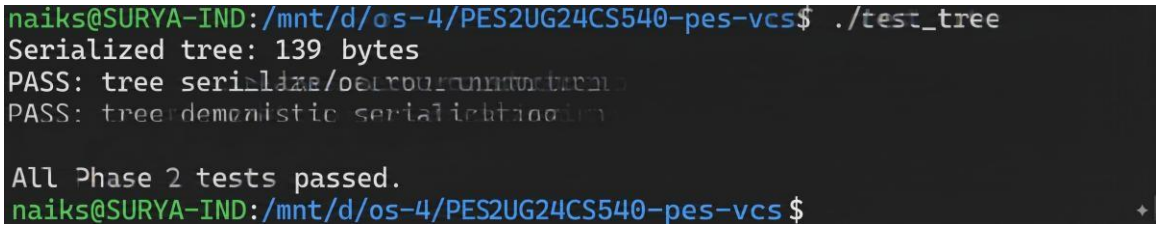
- Tree stored in binary format
- Ensures deterministic hashing
- Entries sorted lexicographically

4. Parsing (tree_parse)

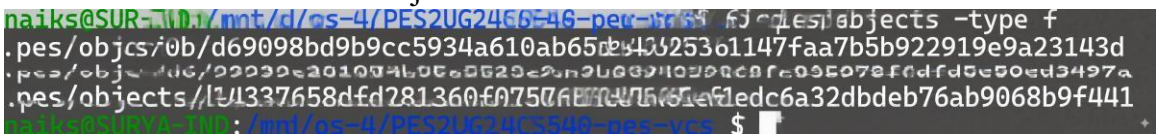
- Converts binary data back into tree structure
- Uses pointer traversal and boundary checks

Phase 2 Verification

Screenshot 2A — Tree Tests



Screenshot 2B — Raw Tree Object



Phase 3: Staging Area (Index)

Objective

To implement an intermediate staging layer before commits.

Implementation Details

1. Index File

- Stored at .pes/index Maintains list of staged files Stores: o File path o Hash o Size o Timestamp o Mode 2. Index Operations index_load

- Loads index from disk
- Initializes empty index if file missing index_save

- Writes index to disk index_add

- Reads file contents
- Creates blob object
- Updates index entry

3. Change Tracking

 Detects:

o Modified files

o Deleted files o Untracked files

Phase 3 Verification

Screenshot 3A — Command Flow

```
naiks@SURYA-IND:/mnt/d/os-4/PES2UG24CS540-pes-vcs$ ./pres add file1.txt file2.txt
naiks@SURYA-IND:/mnt/d/os-4/PES2UG24CS540-pes-vcs$ ./pres status
Staged changes:
  (nothing to show)

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  .gitignore
  untracked:  commit.c
  untracked:  commit.h
  untracked:  file1.txt
  untracked:  file2.txt
  untracked:  index.c
  untracked:  index.h
  untracked:  Makefile
  untracked:  object.c
  untracked:  pes.c
  untracked:  pes.h
  untracked:  README.md
  untracked:  test_objects.c
  untracked:  test_sequence.sh
  untracked:  test_tree.c
  untracked:  tree.c
  untracked:  tree.h

naiks@SURYA-IND:/mnt/d/os-4/PES2UG24CS540-pes-vcs$
```

Screenshot 3B — Index Content

```
naiks@SURYA-IND:/mnt/d/[illegible]
```

Phase 4: Commits and History

Objective

To create snapshots of project state and maintain history.

Implementation Details

1. Commit Structure

Each commit contains:

- Tree hash
- Parent commit hash

- Author
- Timestamp
- Commit message

2. Commit Creation (commit_create)

Steps:

1. Build tree from index
2. Read parent commit from HEAD
3. Create commit object
4. Serialize commit
5. Store commit using object_write
6. Update HEAD reference

3. HEAD Management

- .pes/HEAD → points to branch
- .pes/refs/heads/main → stores latest commit hash

4. Commit History (pes log)

- Traverses commits via parent pointers
- Displays history in reverse order

Phase 4 Verification

Screenshot 4A — Commit Log

```
naiks@SURYA-IND:/mnt/d/os-4/PES2UG24CS540-pes-vcs$ ./pes log
commit 3b64591f9e06844588ce6f0a1a68763993c5cf1984f2c8a49f8645db4965f246
Author: Surya
Date:   1776336389

    Add bye

commit 30b5e3d1b4e1ac289e1477d973ee952228c87511880460f1ed6896c0ad0c135c
Author: Surya
Date:   1776336388

    Add world

Add bye

commit 9d9b5d81f5842ee39f51ef959b54c164fb7eebd73168aefc2e563f9bf880cbcc
Author: Surya
Date:   1776336388

    Initial commit◆◆◆W

naiks@SURYA-IND:/mnt/d/os-4/PES2UG24CS540-pes-vcs$ |
```

Screenshot 4B — Object Growth

```
naiks@SURYA-IND:/mnt/d/os-4/PES2UG24CS540-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/30/b5e3d1b4e1ac289e1477d973ee952228c87511880460f1ed6896c0ad0c135c
.pes/objects/3b/64591f9e06844588ce6f0a1a68763993c5cf1984f2c8a49f8645db4965f246
.pes/objects/48/1407f7237f7bd749dca31e2d8422bafe211741d5556f007efaf821ffc7179
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/9d/9b5d81f5842ee39f51ef959b54c164fb7eebd73168aefc2e563f9bf880cbcc
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/f3/97707e14d1cabf1c29a72d833bffb94d4a474ecc48789d6496742ae6541034
.pes/refs/heads/main
naiks@SURYA-IND:/mnt/d/os-4/PES2UG24CS540-pes-vcs$ |
```

Screenshot 4C — HEAD Verification

```
naiks@SYA-IND:/mnt/d/os-4/PES2UG24CS540-pes-vcs$ cat .pes/refs/heads/main
3b64f9e0444588ce6f1a61a68763993c5cf1984f2c8a49f8645db4965f246
ref: refs/heads/main
```

Final Integration

All modules were integrated into a single executable pes.

The system supports:

- Repository initialization
- File staging
- Commit creation
- History viewing

Final Screenshot:

```
naiks@SURYA-IND:/mnt/d/os-4/PES2UG24CS540_-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: e5a261cbf60e... Initial commit

Log after first commit:
commit e5a261cbf60e79266d272eae66ecfa7a0d30bf82fdd7dfc96a6fce6fe3f493d6
Author: Surya
Date:   1776336663

    Initial commit


--- Second Commit ---
Committed: dd4a6fb0df5d... Update file.txt

--- Third Commit ---
Committed: d5bfe0d5c13a... Add farewell

--- Full History ---
commit d5bfe0d5c13a9d0b1222cb4c0708d2e3d5c64874a19797ef7745a7493831a3f6
```

```
--- Full History ---
commit d5bfe0d5c13a9d0b1222cb4c0708d2e3d5c64874a19797ef7745a7493831a3f6
Author: Surya
Date:   1776336664

    Add farewell

commit dd4a6fb0df5df9cf119b710de9805927577ded9cee27edf8de06fe54a7fa6dde
Author: Surya
Date:   1776336664

    Update file.txt

commit e5a261cbf60e79266d272eae66ecfa7a0d30bf82fdd7dfc96a6fce6fe3f493d6
Author: Surya
Date:   1776336663

    Initial commit◆Z    _


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
d5bfe0d5c13a9d0b1222cb4c0708d2e3d5c64874a19797ef7745a7493831a3f6

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/d5/bfe0d5c13a9d0b1222cb4c0708d2e3d5c64874a19797ef7745a7493831a3f6
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/dd/4a6fb0df5df9cf119b710de9805927577ded9cee27edf8de06fe54a7fa6dde
.pes/objects/e5/a261cbf60e79266d272eae66ecfa7a0d30bf82fdd7dfc96a6fce6fe3f493d6

=== All integration tests completed ===
naiks@SURYA-IND:/mnt/d/os-4/PES2UG24CSS540-pes-vcs$
```

Analysis Questions

Phase 5: Branching and Checkout

Q5.1 Checkout Implementation

- Update HEAD reference
- Replace working directory with target branch snapshot Q5.2 Dirty

Directory Detection

Compare:

- Working Directory vs Index
- Index vs HEAD

Q5.3 Detached HEAD

- HEAD points directly to commit
- Commits are not tied to branch

Phase 6: Garbage Collection

Q6.1 Mark and Sweep

- Mark reachable objects
- Delete unreachable objects

Q6.2 Race Conditions

- GC may delete unlinked objects
- Solution: grace period

Conclusion

This project successfully demonstrates the internal working of a version control system and provides hands-on understanding of Git-like architecture, file systems, and OS-level programming.